Verbale esterno

5 marzo 2026

Autore	Damiano Berti
Verificatore	Giulia Romanato
Approvazione	Nicolò Lattanzio

Indice

1 Registro delle versioni **2**

2 Informazioni introduttive **2**
2.1 Durata e luogo 2
2.2 Partecipanti 2

3 Contenuto della riunione **2**
3.1 Ordine del giorno 2

4 Riassunto della discussione **3**
4.1 Discussione su sezione test del report 3
4.2 Strategia di Deploy su AWS e Gestione dei Costi 3
4.3 Gestione del Database MongoDB 3
4.4 Architettura del Backend in NestJS 3
4.5 Scansione di Sicurezza e Ottimizzazione Analisi LLM nel PoC 3
4.6 Scelta del Framework Frontend (Ecosistema React) 4

5 Decisioni e azioni **4**

1 Registro delle versioni

Versione	Data	Autore	Descrizione delle modifiche
1.0.0	13/03/2026	Nicolò Lattanzio	Approvazione del verbale
0.1.0	09/03/2026	Giulia Romanato	Verifica del verbale
0.0.1	06/03/2026	Damiano Berti	Prima stesura del verbale

2 Informazioni introduttive

2.1 Durata e luogo

- **Inizio:** 14:30
- **Fine:** 15:00
- **Luogo:** Meeting Microsoft Teams

2.2 Partecipanti

Nome e Cognome	Presente	Assente
Damiano Berti	■	☐
Alessandro Frison	■	☐
Lorenzo Grolla	■	☐
Nicolò Lattanzio	■	☐
Alessandro Morabito	■	☐
Giacomo Nalotto	■	☐
Giulia Romanato	■	☐

3 Contenuto della riunione

3.1 Ordine del giorno

1. Discussione su sezione test del report
2. Strategia di deploy su AWS e gestione dei costi
3. Gestione del database MongoDB
4. Architettura del backend in NestJS
5. Scansione di sicurezza e ottimizzazione analisi LLM nel PoC
6. Valutare utilizzo di un framework frontend basato su React rispetto a React puro

4 Riassunto della discussione

4.1 Discussione su sezione test del report

Si è discussa l'implementazione dell'analisi dei test, definendo la "qualità di un test" in base a quanto sia ben scritto e alla sua capacità di coprire efficacemente il codice in esame. Si è stabilito che la metrica principale di riferimento rimane la code coverage. Su indicazione dell'azienda, il raggiungimento di una determinata code coverage sarà un requisito obbligatorio per il rilascio, mentre l'analisi qualitativa approfondita dei test verrà trattata come un obiettivo desiderabile (opzionale).

4.2 Strategia di Deploy su AWS e Gestione dei Costi

Si è affrontato il tema delle tempistiche e dell'ottimizzazione dei costi relativi al deploy su infrastruttura AWS. La decisione finale è di adottare Amazon ECS (Elastic Container Service) fin dalle prime fasi della progettazione. Questa scelta garantisce costi contenuti durante la fase di testing iniziale e fornisce una base solida da valutare anche per il deploy in ambiente di produzione.

4.3 Gestione del Database MongoDB

È stata valutata l'opportunità di utilizzare Amazon DocumentDB come alternativa a un'istanza MongoDB tradizionale. L'ipotesi è stata scartata poiché DocumentDB non riceve aggiornamenti con la stessa frequenza di MongoDB, rischiando di causare disallineamenti sulle feature più recenti. Si è deciso quindi valutare due opzioni:

- L'utilizzo di MongoDB Atlas (per il quale sarà necessario ricevere le credenziali di accesso fornite dall'azienda).
- L'utilizzo di un'istanza MongoDB hostata tramite ECS su AWS.

4.4 Architettura del Backend in NestJS

Il dibattito si è concentrato sulla possibile adozione di un'architettura esagonale per il backend sviluppato in NestJS. Valutando il rapporto costi-benefici, si è discusso sul fatto che affinchè l'utilizzo di una architettura esagonale sia giustificato, deve essere presente un chiaro problema che per essere risolto richieda questa architettura.

4.5 Scansione di Sicurezza e Ottimizzazione Analisi LLM nel PoC

È stato analizzato il report generato dal Proof of Concept relativo alla scansione del repository. Un focus particolare è stato dedicato alla sicurezza, in specifico alle vulnerabilità identificate da Semgrep. Si è discusso se effettuare una singola invocazione a un LLM per ogni vulnerabilità, fornendo lo snippet di codice corrispondente per ottenere una remediation mirata. Per ottimizzare tempi e costi, sono emerse due strategie da valutare:

- Raggruppare problemi simili in un'unica chiamata, riducendo il numero totale di richieste.
- Eseguire le interrogazioni in parallelo, sfruttando le funzionalità di LangGraph per lanciare istanze simultanee dei nodi.

4.6 Scelta del Framework Frontend (Ecosistema React)

Si intendeva discutere l'adozione di un framework basato su React rispetto all'utilizzo di React puro. Tuttavia, a causa dell'assenza dei referenti aziendali per l'area frontend, non è stato possibile prendere decisioni in merito.

5 Decisioni e azioni

Codice	Descrizione	Assegnatario
DEC-PB-001	Includere nell'mvp la code coverage, invece rimandata la decisione se includere anche la qualità dei test nei report	Tutti
DEC-PB-002	utilizzare ECS per il deployment	Tutti
DEC-PB-003	Non utilizzare DocumentDB, ma rimandata la decisione se usare MongoDB Atlas o hostare un'istanza MongoDB su AWS	Tutti
DEC-PB-004	Utilizzare un'architettura layered per il backend nestjs	Tutti
DEC-PB-005	Rimandata la scelta sull'architettura della scansione di sicurezza, da valutare alla luce dei nuovi spunti emersi	Tutti
DEC-PB-006	Rimandata la decisione sull'adozione di un framework frontend basato su React	Tutti
AZ-PB-001	Ragionare sulle decisioni da prendere rimandate e provare ad abbozzare una architettura backend layered	Tutti

Firma del proponente